Aengus McGuinness

(805) 570-1436 | amcguinness@college.harvard.edu | github.com/AengusMcGuinness | aengusmcguinness.com

Education

Harvard College Cambridge, MA
A.B. Computer Science — GPA 3.88 / ACT 35, John Harvard Scholar *Expected May 2027*

- **Completed:** Distributed Computing, Operating Systems, Modern Storage Systems, Computer Architecture, Systems Programming & Machine Organization, Computing Hardware, Machine Learning, Algorithms, Probability
- **Fall 2026:** Computing at Scale, Advanced Computer Networks, Advanced Topics in Computer Architecture, Systems Security, Modern AI Systems

Experience

Ironsite San Francisco, CA
ML Infrastructure Engineering Intern — 51 PRs in 10 weeks *June 2026 – Aug 2026*

- Built and productionized the company's GPU inference platform — a self-hosted multimodal LLM on vLLM over autoscaled GCP spot fleets, ∼10k lines of Terraform/Packer/Ansible — completing **61k+ production video inferences** in five weeks; pre-hydrated weight disks cut per-worker model load **8**× (7 min → 53 s).
- Colocated the inference engine inside each 8-GPU worker, deleting the load-balancer tier; gated the cutover on prompt-token parity against the live fleet and a crash drill proving exactly-once processing under spot preemption.
- Grew production ML throughput **3.3**× in five weeks (4.7k → 15.3k videos/week) while raising end-of-day pipeline completion from ∼80% to **97–99.8%** on 60% higher daily intake.
- Designed cross-cloud data-residency enforcement (US-only vs. global) across **7 compute providers**: per-region queue pairs, fail-closed verification of provider-reported placement, and IAM session-policy denial — unblocking a major enterprise customer, zero placement violations at rollout.
- Hardened capacity acquisition and cost control: zonal fallback for spot stockouts (a 156-retry regional stall became first-try placement), metric-based idle-GPU detection bounding a ∼$1.3k/incident burn to ∼15 min, MIG autohealing, and preemption alerting that absorbed **254 spot preemptions**.

Harvard Generative AI Research Program Cambridge, MA
Research Fellow *June 2025 – Aug 2025*

- Authored the SLURM automation toolkit for a GPU protein-structure refinement pipeline (OpenFold/ROCKET on A100s) — MSA generation and clustering, X-ray data prep, config-driven refinement sweeps, scoring — turning a hand-run multi-day workflow into reproducible one-command job submissions now used by the lab.

Los Alamos National Laboratory, B-GEN Santa Fe, NM
Student Researcher *Summers 2022 – 2024*

- Cut terabyte-scale genomics pipeline runtime ∼80% via SLURM parallelization, memory tuning, and GPU offload of throughput-bound stages.

Systems & Performance Projects

RCache — One-Sided RDMA Distributed Cache | *C++, libibverbs* | code Solo · Spring 2026

- Built a key-value cache with three transports over `libibverbs`: a TCP/RPC baseline, two-sided RDMA send/recv, and one-sided reads served straight out of a registered hash-table memory region, with an optional RDMA `FETCH_AND_ADD` maintaining LRU recency metadata without server CPU involvement.
- Across three CloudLab trials on Mellanox hardware, one-sided reads sustained **974k ops/s** at 8 clients with a flat **12 μs p99** — **13**× the throughput and **13**× lower tail latency than TCP (157 μs p99) — and isolated the metadata atomic's cost at 3–4 μs p99: recency tracking stays on the critical path even when reads leave it.

Chickadee OS Kernel | *C++, x86-64* Spring 2026

- Implemented blocking system calls (`sys_waitpid`, `sys_msleep`) with a timer-driven sleep mechanism backed by a hashed timing wheel, removing busy-wait loops while preserving correctness under multi-core preemption.

Stream Buffer Prefetcher | *C++, Intel Pin* | code Team of 4 · Spring 2026

- Built a Pin-based memory-hierarchy simulator implementing Jouppi's fixed-depth stream buffer; sweeping depth × stream count over SPEC CPU2006 showed `libquantum`'s effective L1D misses collapse **43**× while irregular workloads gain ∼2% for up to **9**× the L2 traffic — the bandwidth-waste finding that motivated the team's adaptive-depth follow-on.
- Wrote the Pin harness and resume-on-kill parallel sweep driver (72 configs) used to evaluate that adaptive policy: **5.02**× speedup on `libquantum`, prefetch accuracy 78% → **83%** on `dealII`.

Technical Skills

Languages: C++, Rust, Python, Bash, SQL
Systems: Linux, concurrency, distributed systems, low-latency networking, RDMA (`libibverbs`), kernel development, Intel Pin
Infrastructure: AWS, GCP, Terraform/OpenTofu, Ansible, Docker Swarm, SLURM, vLLM, gRPC, Prometheus, Postgres